COMP26120

Notes on Algorithmic Complexity

Francisco Lobo

Academic Session: 2025-26

Introduction

These notes were written to introduce the material on algorithmic complexity for Comp26120 course unit, which the standard textbooks do not cover in sufficient mathematical detail. The current presentation is also supported by the Comp26120 video resources for the complexity topic, which provide further explanation and examples. You can find an alternative presentation and additional examples & exercises in the recommended textbooks.

- Algorithm Design and Applications by Goodrich and Tamassia has a concise presentation of the required topics in chapter 1.

  The relevant sections are 1.1 on the analysis of algorithms (16 pages) and 1.3 with detailed a case study (5 pages). You may also find useful the review of relevant mathematical background in section 1.2.

- Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein covers the material at a slower pace in chapters 2 and 3.

  Section 2.1 gives an analysis of the insertion sort algorithm, similar to the one given in the videos for this week. Section 2.2 introduces a generic uniprocessor, random-access machine model of computation as the implementation technology for the analysis of algorithms. Section 3.1 discusses the order of growth of an algorithm's running time and section 3.2 formally introduces the relevant complexity classes.

Intended Learning Outcomes

- Understand fundamental sorting and searching algorithms
- Understand the generic random-access machine model of computation
- Develop best and worst complexity estimates for a given algorithm
- Understand the big-Ω, big-O, and big-Θ complexity classes
- Establish if a complexity estimate is in a particular complexity class
- Analyse the running time taken by a given algorithm
- Establish lower, upper, and tight asymptotic bounds for a given algorithm
- Understand the asymptotic behaviour of big-Ω, big-O, and big-Θ
- Understand the asymptotic growth of polynomial and exponential complexities
- Establish asymptotic properties of big-Ω, big-O, and big-Θ

These notes include a set of 20 exercises which you are expected to work through. Some of them are selected as core exercises for you to complete as part of the formative Comp26120 Worksheet 2.

1. Algorithmic Complexity Classes

Motivating Example

When considering the performance of an algorithm we are primarily interested in either the time it takes to run, or the memory space it uses during execution. We will focus on running time performance, although memory space may also be critical; in particular for embedded systems and hardware applications.

Example 1 (Bubble Sort)**.** The bubble sort algorithm for putting a list (array) of integers `A` of size `n` in ascending order is implemented by the following C language code.

```
1  void bubble_sort(int *A, int n) {
2    for (int i = 0; i < n-1; i++) {
3      for (int j = 0; j < n-1; j++) {
4        if (A[j] > A[j+1]) {
5          int tmp = A[j];
6          A[j] = A[j+1];
7          A[j+1] = tmp;
8        }
9      }
10   }
11 }
```

The code in lines 5–7 simply swaps the array values in positions `j` and `j+1`.

What is the running time cost of bubble sort? For simplicity, we may assume that each instruction takes unit time to execute, so we just need to count how many times each instruction is run. Each of the two loops makes $n-1$ iterations, but the loop instruction is also executed when its condition turns false, hence line 2 runs $n-1+1$ times and line 3 runs $(n-1)(n-1+1)$ times. The following table summarises the calculations.

Line	2	3	4	5	6	7
Runs	n	$n(n-1)$	$(n-1)^2$	$S(n)$	$S(n)$	$S(n)$

The number of times $S(n)$ that the lines of the swap code are run will depend on the initial order of values in the array. The most important of possible situations are the following two estimates for the total running time cost $T(n)$ of bubble sort.

Best Case:

In the best case, the array is already in ascending order and the `if` condition is always false, meaning $S_{\min}(n) = 0$ is constant and hence

$$\begin{aligned} T_{\min}(n) &= n + n(n-1) + (n-1)^2 + 3 \cdot 0 \\ &= 2n^2 - 2n + 1 \end{aligned}$$

Worst Case:

In the worst case, the array is initially in descending order and the `if` condition is always true, meaning $S_{\max}(n) = (n-1)^2$ and hence

$$\begin{aligned} T_{\max}(n) &= n + n(n-1) + (n-1)^2 + 3 \cdot (n-1)^2 \\ &= 5n^2 - 8n + 4 \end{aligned}$$

Overall, the running time behaviour of bubble sort will always be quadratic with respect to the size of the array. This observation suggests that the complexity of this algorithm is formally $\Theta(n^2)$ as defined below.

Complexity Estimates

In order to define the running time (or memory space) performance of an algorithm, we must fix the model of computation governing its execution. For our purposes, we can assume that algorithms are programs for a generic *random-access machine*, whose main characteristics are:

1. All memory is equally expensive to access and instructions executed sequentially one after another (i.e. there are no concurrent operations).
2. All primitive instructions take unit time to execute, and the time taken by a function call is the total of primitive instructions involved.
3. All primitive data types are implemented using a constant word size, i.e. a fixed number of bits, and the space used by an array type is the total of words required for its length.

To formally describe a random-access machine, we would need to specify exactly which are its primitive instructions and data types. However, in the analysis of general algorithms it is more useful for the concept of "primitive" to vary. For example, when comparing integer exponentiation algorithms we may only allow basic arithmetic operations as primitive, but if discussing algorithms for prime factorisation we may legitimately regard some forms of exponentiation as primitive. Similarly, when analysing graph algorithms we may consider that graph edges can be stored in unit space, regardless of how much memory these would use in a real world implementation.

Note that when discussing the complexity of *computational problems* themselves, at the end of semester 2, it will be necessary to fix a rigorous notion of computational model, such as a *Turing machine*. But it is reasonable not to fix the formal details of random-access machines, as these will not impact the comparison of asymptotic complexity of *algorithms* using the same primitive instructions and data types. In this part of the course, we will present algorithms in either pseudo-code or in the C programming language (whose syntax is much like that of Java but has simpler data types).

Definition 1. Let $I_A \subseteq \mathbb{N}$ be the set of valid sizes for the input data of the algorithm A. A *complexity estimate* for A is a non-negative real valued function $f\colon I_A \to \mathbb{R}_{\geq 0}$. This estimate provides a *lower or upper bound* for either the running time or memory space used on a suitable random-access machine by the algorithm.

In a *best case analysis* f is a lower bound, meaning that for all valid inputs of size $n \in \mathbb{N}$ the time or space required by A on that input is *at least* $f(n)$. In a *worst case analysis* f is an upper bound, meaning that for all valid inputs the time or space required by A on an input of size $n \in \mathbb{N}$ is *at most* $f(n)$.

Note that the technical detail limiting a complexity estimate $f(n)$ for an algorithm A to only valid sizes $I_A \subseteq \mathbb{N}$ of its input data is only relevant in special applications. We will generally assume that complexity measures are defined for all natural numbers $n \in \mathbb{N}$ unless otherwise stated. Moreover, it will be useful to extend the domain of definition of a complexity measure $\mathbb{N} \to \mathbb{R}_{\geq 0}$ to all real values; in particular, this will often be implicit when the estimate is given by a real valued algebraic form.

As in our previous motivating example, we will be focusing on running time estimates. Under the assumptions for random-access machines, this corresponds to the total number of primitive steps the algorithm executes. How we characterise "input size" will depend on the kind of algorithm: for sorting lists the relevant metric is the number of list items, for graph operations the total number of graph nodes and/or edges, for multiplication of binary numbers the total number of binary bits, etc.

Asymptotic Complexity

Once we have a complexity estimate $f(n)$ for a given algorithm A we are mostly interested on how $f(n)$ behaves for large input sizes, as this is what distinguishes A from alternative algorithms. In mathematical terms, this means we want to focus on the *asymptotic behaviour* of $f(n)$ as $n \to \infty$, and to this end we will introduce various *complexity classes* (sets) of functions with the same asymptotic behaviour. This will allow us to classify the performance of algorithms in accordance with which complexity class it belongs to.

Definition 2. Suppose g is a complexity estimate.

1. The complexity class $\Omega(g)$ is the set of all estimate functions f with the following property.

$$f \in \Omega(g) \quad \Longleftrightarrow \quad \begin{array}{l}\text{There exist constants } c > 0 \text{ and } n_0 > 0 \\ \text{such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0.\end{array}$$

   We call $\Omega(g)$ the class *big-Ω of g* and say g is a *lower asymptotic bound* of each $f \in \Omega(g)$.

2. The complexity class $\mathrm{O}(g)$ is the set of all estimate functions f with the following property.

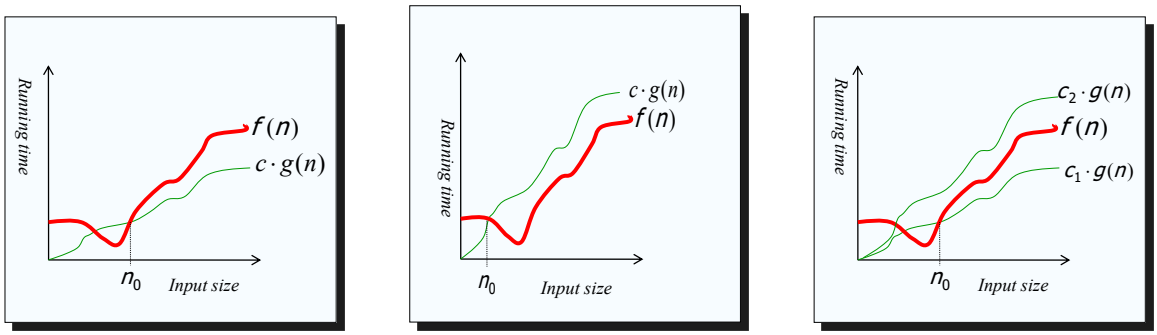
$$f \in \mathrm{O}(g) \quad \Longleftrightarrow \quad \begin{array}{l}\text{There exist constants } c > 0 \text{ and } n_0 > 0 \\ \text{such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0.\end{array}$$

   We call $\mathrm{O}(g)$ the class *big-O of g* and say g is an *upper asymptotic bound* of each $f \in \mathrm{O}(g)$.

3. The complexity class $\Theta(g)$ is the set of all estimate functions f with the following property.

$$f \in \Theta(g) \quad \Longleftrightarrow \quad \begin{array}{l}\text{There exist constants } c_1 > 0,\ c_2 > 0, \text{ and } n_0 > 0 \\ \text{such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0.\end{array}$$

   We call $\Theta(g)$ the class *big-Θ of g* and say g is a *tight asymptotic bound* of each $f \in \Theta(g)$.



(a) When f is $\Omega(g)$ (b) When f is $\mathrm{O}(g)$ (c) When f is $\Theta(g)$

Figure 1: Behaviour of estimate f in various complexity classes of function g.

We often treat big-Ω, big-O, and big-Θ of an estimate g as properties that another estimate f may or not have, thus we usually write e.g. f is $\mathrm{O}(g)$ instead of the more formal f *is in* $\mathrm{O}(g)$.

It is important to observe that the underlying predicate that defines the above complexity classes is the notion of "eventual domination" that was introduced in Comp11120. For general real functions, we say that $h(x)$ *eventually dominates* $f(x)$ precisely when there is r such that $f(x) \leq h(x)$ for all $x \geq r$. Using this concept, $\mathrm{O}(g)$ could be defined as the class of estimate functions f for which there is $c > 0$ such that $c \cdot g$ eventually dominates f. This is how the big-O class was defined in Comp11212. Similarly, $\Omega(g)$ is the class of estimates f which eventually dominate some positive scaling of g. For f to be $\Theta(g)$ both of these situations must arise, as clarified by the following proposition.

Proposition 3. *For all complexity estimates f and g, $f \in \Theta(g)$ if and only if $f \in \Omega(g) \cap \mathrm{O}(g)$. Moreover $\Theta(g) = \Omega(g) \cap \mathrm{O}(g)$ holds for all estimates g.*

Proof. Suppose $f \in \Theta(g)$. In this case there is $c_1 > 0$ and $n_0 > 0$ such that $0 \leq c_1 \cdot g(n) \leq f(n)$ for all $n \geq n_0$ and hence $f \in \Omega(g)$. But there is also $c_2 > 0$ and $n_0 > 0$ such that $0 \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$ and thus $f \in \mathrm{O}(g)$. It follows that f is in the intersection $\Omega(g) \cap \mathrm{O}(g)$ and the forward implication holds.

Now consider the backward implication. As $f \in \Omega(g)$ there is $c' > 0$ and $n'_0 > 0$ such that $0 \leq c' \cdot g(n) \leq f(n)$ for all $n \geq n'_0$. Since $f \in \mathrm{O}(g)$ there is also $c'' > 0$ and $n''_0 > 0$ such that $0 \leq f(n) \leq c'' \cdot g(n)$ for all $n \geq n''_0$. If we define $n_0 = \max\{n'_0, n''_0\}$ it follows that $0 \leq c' \cdot g(n) \leq f(n) \leq c'' \cdot g(n)$ for all $n \geq n_0$. Thus we have complexity witnesses c', c'', and n_0 that establish $f \in \Theta(g)$. □

There is also an underlying symmetry in the definitions of the Ω, O, and Θ complexity classes, which is made formal in the following proposition.

Proposition 4 (Symmetry)**.** *For all complexity estimates f and g the following hold.*

(i) f is $\mathrm{O}(g)$ if and only if g is $\Omega(f)$

(ii) f is $\Theta(g)$ if and only if g is $\Theta(f)$

Proof. Consider the 1st statement. Suppose $f \in \mathrm{O}(g)$ hence there is $c > 0$ and $n_0 > 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Multiplying by $c^{-1} > 0$ it follows that

$$0 \leq c^{-1} \cdot f(n) \leq c^{-1} \cdot c \cdot g(n) = g(n)$$

for all $n \geq n_0$. Thus we have witnesses n_0 and c^{-1} showing $g \in \Omega(f)$. This justifies the forward implication and similar argument validates the opposite direction.

Consider the 2nd statement. By Proposition 3 we have f is $\Theta(g)$ if and only if both $f \in \Omega(g)$ and $f \in \mathrm{O}(g)$. Using the 1st statement the latter are equivalent to $g \in \mathrm{O}(f)$ and also $g \in \Omega(g)$ respectively, which together hold precisely when g is $\Theta(f)$. □

We will develop further properties of the big-Ω, big-O, and big-Θ classes after some examples. Note that the literature defines additional complexity classes, in particular *little-ω* and *little-o*, which we will not consider here because these are less connected to best or worst case analyses of algorithms.

Continued Example

When estimating the running time cost of an algorithm, a best case analysis determines the big-Ω class for that algorithm, and a worst case analysis determines its big-O class. Consider again our previous discussion of bubble sort. We have claimed that this algorithm is formally $\Theta(n^2)$. Using Proposition 3, we can establish this fact by showing that bubble sort is both $\Omega(n^2)$ and $\mathrm{O}(n^2)$. This follows from our previous best and worst case analyses.

Best Case:

This analysis obtained the estimate $T_{\min}(n) = 2n^2 - 2n + 1$ as a lower bound for the running time of bubble sort. To show that this algorithm is $\Omega(n^2)$ we must find complexity witnesses c and n_0 that establish $T_{\min} \in \Omega(n^2)$.

It is clear that $4n - 2 \geq 0$ for all $n \geq 0.5$ so we can fix $n_0 = 1$ knowing that by the derivative test $T_{\min}$ is monotonically increasing for $n \geq n_0$. The big-Ω constraint gives

$$c \cdot n^2 \leq 2n^2 - 2n + 1 \quad \iff \quad c \leq 2 - \frac{2}{n} + \frac{1}{n^2}$$

Note that we are allowed to divide by n^2 and keep the inequality in place because $n \geq 1$. We find that $2\,n^{-2} - 2\,n^{-3} \geq 0$ when $n \geq 1$ so by the derivative test $T_{\min}(n)/n^2$ is monotonically increasing for $n \geq n_0$. Moreover

$$\frac{T_{\min}(1)}{1^2} = 1 \qquad\qquad \lim_{n\to\infty} \frac{T_{\min}(n)}{n^2} = 2$$

Thus we can choose any $c \leq 1$. In particular, $c = 1$ and $n_0 = 1$ witness that $T_{\min}$ is $\Omega(n^2)$.

Worst Case:

This analysis obtained the estimate $T_{\max}(n) = 5n^2 - 8n + 4$ as an upper bound for the running time of bubble sort. To show that this algorithm is $\mathrm{O}(n^2)$ we must find complexity witnesses c and n_0 that establish $T_{\max} \in \mathrm{O}(n^2)$.

It is clear that $10n - 8 \geq 0$ for all $n \geq 0.8$ so we can fix $n_0 = 1$ knowing that by the derivative test $T_{\max}$ is monotonically increasing for $n \geq n_0$. The big-O constraint gives

$$5n^2 - 8n + 4 \leq c \cdot n^2 \quad \iff \quad 5 - \frac{8}{n} + \frac{4}{n^2} \leq c$$

Again note that as $n \geq 1$ we can divide by n^2 and not change the inequality. We find that $8\,n^{-2} - 8\,n^{-3} \geq 0$ when $n \geq 1$ so by the derivative test $T_{\max}(n)/n^2$ is monotonically increasing for $n \geq n_0$. Moreover

$$\frac{T_{\max}(1)}{1^2} = 1 \qquad\qquad \lim_{n\to\infty} \frac{T_{\max}(n)}{n^2} = 5$$

Thus we can choose any $c \geq 5$. In particular, $c = 5$ and $n_0 = 1$ witness that $T_{\max}$ is $\mathrm{O}(n^2)$.

Another Example

We now consider the insertion sort algorithm. You can watch a step by step run of it on a sample array in the Comp26120 videos for the complexity topic.

Example 2 (Insertion Sort)**.** The insertion sort algorithm for putting a list (array) of integers `A` of size `n` in ascending order is implemented by the following C language code.

```
1  void insertion_sort(int *A, int n) {
2    for (int i = 1; i < n; i++) {
3      int key = A[i];
4      int j = i-1;
5      while (j > 0 && A[j] > key) {
6        A[j+1] = A[j];
7        j--;
8      }
9      A[j+1] = key;
10   }
11 }
```

The inner loop in lines 5–8 shifts up all items between positions `i-1` and `0` that are greater than `key` so this value can be put in the correct place.

To calculate the running time cost of insertion sort we must count how many times each instruction is executed. When $n = 0$ or $n = 1$ the `for` loop condition is false, so line 2 runs once but the loop body doesn't run. Otherwise, the outer loop makes $n - 1$ iterations, and line 2 runs one more time when its condition turns false. The following table summarises the case when $n > 1$.

Line	2	3	4	5	6	7	9
Runs	n	$n-1$	$n-1$	$\sum_{i=1}^{n-1} 1 + S(i)$	$\sum_{i=1}^{n-1} S(i)$	$\sum_{i=1}^{n-1} S(i)$	$n-1$

The number of iterations $S(i)$ that the `while` loop makes depends on the value of the outer loop variable i which ranges from 1 to $n-1$ and the numbers in the array. Thus the estimate for the total running time cost of insertion sort is

$$T(n) = n + 3 \cdot (n-1) + \left(\sum_{i=1}^{n-1} 1 + S(i) \right) + 2 \cdot \left(\sum_{i=1}^{n-1} S(i) \right)$$

We now consider best and worst cases of `while` loop execution, and establish asymptotic lower and upper bounds for this algorithm.

Best Case:

If the array is already in ascending order, then the condition `A[j]>key` is always false, meaning $S_{\min}(i) = 0$ is constant. In this case

$$\begin{aligned} \sum_{i=1}^{n-1} 1 + S_{\min}(i) &= \sum_{i=1}^{n-1} 1 \\ &= n-1 \end{aligned} \qquad \begin{aligned} \sum_{i=1}^{n-1} S_{\min}(i) &= \sum_{i=1}^{n-1} 0 \\ &= 0 \end{aligned}$$

Hence the lower bound for the running time of insertion sort is

$$\begin{aligned} T_{\min}(n) &= n + 3 \cdot (n-1) + (n-1) + 2 \cdot 0 \\ &= 5n - 4 \end{aligned}$$

We now establish that this algorithm is $\Omega(n)$. It is clear that $T_{\min}$ is monotonically increasing since its derivative is $5 \geq 0$. So we can fix $n_0 = 1$. The big-Ω constraint gives

$$c \cdot n \leq 5n - 4 \quad \iff \quad c \leq 5 - \frac{4}{n}$$

We observe that $T_{\min}(n)/n$ is well defined for $n \geq 1$ and also monotonically increasing because it has a non-negative derivative $4\,n^{-2} \geq 0$ in all its domain. Moreover

$$\frac{T_{\min}(1)}{1} = 1 \qquad \lim_{n \to \infty} \frac{T_{\min}(n)}{n} = 5$$

Thus we can choose any $c \leq 1$. In particular, $c = 1$ and $n_0 = 1$ witness $T_{\min}$ is $\Omega(n)$.

Worst Case:

If the array is initially in descending order, then the condition `A[j]>key` is always true, and the inner loop executes for values of j ranging from $i-1$ to 1. This means $S_{\max}(i) = i - 1$ is an arithmetic progression. It follows that

$$\begin{aligned} \sum_{i=1}^{n-1} 1 + S_{\max}(i) &= \sum_{i=1}^{n-1} i \\ &= \frac{(n-1)}{2} \cdot (1 + n - 1) \\ &= \frac{(n-1)n}{2} \end{aligned} \qquad \begin{aligned} \sum_{i=1}^{n-1} S_{\max}(i) &= \sum_{i=1}^{n-1} i - 1 \\ &= \frac{(n-1)}{2} \cdot (0 + n - 2) \\ &= \frac{(n-1)(n-2)}{2} \end{aligned}$$

Hence the upper bound for the running time of insertion sort is

$$\begin{aligned} T_{\max}(n) &= n + 3 \cdot (n-1) + \frac{(n-1)n}{2} + 2 \cdot \frac{(n-1)(n-2)}{2} \\ &= \frac{3}{2}n^2 + \frac{1}{2}n - 1 \end{aligned}$$

We now establish that this algorithm is $\mathrm{O}(n^2)$. We find that $T_{\max}$ is monotonically increasing as its derivative $3n + \frac{1}{2}$ is non-negative for $n \geq 0$. So we fix $n_0 = 1$. The big-O

constraint gives

$$\frac{3}{2}n^2 + \frac{1}{2}n - 1 \leq c \cdot n^2 \qquad \Longleftrightarrow \qquad \frac{3}{2} + \frac{1}{2n} - \frac{1}{n^2} \leq c$$

We observe that $T_{\max}(n)/n^2$ is well defined for $n \geq 1$. Its derivative $-\frac{1}{2}\,n^{-2} + 2\,n^{-3}$ has a zero at $n = 4$, is positive when $n < 4$, and is negative when $n > 4$. It follows that $n = 4$ is a maximum of $T_{\max}(n)/n^2$; moreover

$$\frac{T_{\max}(1)}{1^2} = 1 \qquad \frac{T_{\max}(4)}{4^2} = 1.5625 \qquad \lim_{n\to\infty} \frac{T_{\max}(n)}{n^2} = 1.5$$

Thus we can choose any $c \geq 1.5625$. In particular, $c = 2$ and $n_0 = 1$ witness $T_{\max}$ is $\mathrm{O}(n^2)$.

Exercise Activities

Exercise 1. Consider sorting the array $A = [31, 41, 59, 26, 41, 58]$ with $n = 6$ elements. Illustrate the operation of the sorting algorithms of Examples 1 and 2 by constructing a table that shows the state of the array at the end of the first n iterations of the algorithm, which in the case of bubble sort will be done by the inner `for` loop.

	Bubble Sort						Insertion Sort					
Iteration	$A[0]$	$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$	$A[0]$	$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$
1st												
2nd												
3rd												
4th												
5th												
6th												

Exercise 2. Apply the big-O definition to each of the following complexity estimates for inputs with size $n \in \mathbb{N}$ and establish whether these are $\mathrm{O}(n^2)$.

(i) $T(n) = 3n^2 - 5n + 4$

(ii) $S(n) = n^3 - 7n^2 + 15n + 2$

(iii) $R(n) = n^2 + 2n - 3n \log n + 5$ for $n > 0$

Determine strict complexity witnesses if the estimate is in the $\mathrm{O}(n^2)$ complexity class, or otherwise justify why these cannot exist.

Exercise 3. Apply the big-Ω definition to each of the following complexity estimates for inputs with size $n \in \mathbb{N}$ and establish whether these are $\Omega(n^2)$.

(i) $T(n) = 3n^2 - 5n + 4$

(ii) $S(n) = n^3 - 7n^2 + 15n + 2$

(iii) $R(n) = n^2 + 2n - 3n \log n + 5$ for $n > 0$

Determine strict complexity witnesses if the estimate is in the $\Omega(n^2)$ complexity class, or otherwise justify why these cannot exist.

Exercise 4. Determine if a tight asymptotic bound exists for each of the following complexity estimates.

(i) $T(n) = \dfrac{n^2}{3} - 2n + 5$

(ii) $S(n) = \dfrac{n^3}{32} - n^2 + 8n + 4$

(iii) $R(n) = n - \log n$ for $n > 0$

Obtain strict complexity witnesses for the corresponding big-Θ class if a tight bound exists, or otherwise justify why it cannot exist.

Exercise 5. We can extend asymptotic complexity classes to the case of two parameters n and m that can go to infinity independently at different rates. For a given function $g(n, m)$, we denote by $\mathrm{O}(g(n, m))$ the set of functions as follows

$$\mathrm{O}(g(n,m)) = \left\{ f(n,m) \;\middle|\; \begin{array}{l} \text{there exist positive constants } c,\ n_0, \text{ and } m_0 \text{ such that} \\ 0 \le f(n,m) \le cg(n,m) \text{ for all } n \ge n_0 \text{ or } m \ge m_0 \end{array} \right\}$$

Give corresponding definitions for $\Omega(g(n, m))$ and $\Theta(g(n, m))$.

We now consider two important search algorithms in common use. You can watch sample runs of these procedures in the Comp26120 videos for the complexity topic.

Example 3 (Linear Search)**.** The linear search algorithm for finding the position of a value `v` in an array of integers `A` with size `n` is implemented by the following C language code.

```c
int linear_search(int v, int *A, int n) {
  int i = 0;
  while (i < n) {
    if (A[i] == v)
      return i;
    i++;
  }
  return -1;
}
```

The implementation returns `-1` if `v` is not found.

Example 4 (Binary Search)**.** The binary search algorithm for finding the position of a value `v` in a *sorted* array of integers `A` with size `n` is implemented by the following C language code.

```c
int binary_search(int v, int *A, int n) {
  int lhs = 0;
  int rhs = n-1;
  while (lhs <= rhs) {
    int cut = (lhs+rhs) / 2;
    if (A[cut] == v)
      return cut;
    else if (A[cut] > v)
      rhs = cut-1;
    else
      lhs = cut+1;
  }
  return -1;
}
```

The implementation returns `-1` if `v` is not found. Note that integer division used in line 5 truncates towards 0 in the C language; thus it returns the floor of the quotient for integers with the same sign, and the ceiling of the quotient otherwise.

Exercise 6. Using best and worst case analyses develop lower and upper bound estimates for the running time of the linear search algorithm from Example 3 in terms of the size n of the array; without ignoring any constant number of steps. Apply the definition of the complexity classes to your estimates in order to show that linear search is both $\Omega(1)$ and $\mathrm{O}(n)$.

Exercise 7. Using best and worst case analyses develop lower and upper bound estimates for the running time of the binary search algorithm from Example 4 in terms of the size n of the array; without ignoring any constant number of steps. Apply the definition of the complexity classes to your estimates in order to show that binary search is both $\Omega(1)$ and $\mathrm{O}(\log n)$.

2. Properties of Algorithmic Complexity

As the previous examples indicate, demonstrating that an algorithm is in a particular big-Ω or big-O class using their definition can involve a long mathematical argument. For this reason, it is useful to establish a few general results that provide intuition and some ready-made complexity assertions.

Proposition 5 (Reflexivity)**.** *For each complexity estimate f the following hold.*

(i) f *is* $\Omega(f)$

(ii) f *is* $\mathrm{O}(f)$

(iii) f *is* $\Theta(f)$

Proof. Since f is a complexity measure we know that it is non-negative. It is then straightforward to confirm that setting all the required witnesses to 1 validates each of these statements. □

Proposition 6 (Ordering)**.** *For all complexity estimates f and g the following hold.*

(i) f *is* $\Omega(g)$ *if and only if* $\Omega(f) \subseteq \Omega(g)$

(ii) f *is* $\mathrm{O}(g)$ *if and only if* $\mathrm{O}(f) \subseteq \mathrm{O}(g)$

(iii) f *is* $\Theta(g)$ *if and only if* $\Theta(f) = \Theta(g)$

Proof. Consider the 1st statement. Suppose $f \in \Omega(g)$ and hence there is $c' > 0$ and $n_0' > 0$ such that $0 \leq c' \cdot g(n) \leq f(n)$ for all $n \geq n_0'$. Fix an arbitrary $e \in \Omega(f)$. Necessarily there is $c'' > 0$ and $n_0'' > 0$ such that $0 \leq c'' \cdot f(n) \leq e(n)$ for all $n \geq n_0''$. It follows that

$$0 \leq c'' \cdot c' \cdot g(n) \leq c'' \cdot f(n) \leq e(n)$$

for all $n \geq n_0 = \max\{n_0', n_0''\}$. Thus we have witnesses n_0 and $c = c'' \cdot c'$ showing $e \in \Omega(g)$ also. Since e was arbitrary $\Omega(f) \subseteq \Omega(g)$ and the forward implication holds. To justify the opposite direction we use Proposition 5 to deduce that $f \in \Omega(f) \subseteq \Omega(g)$ which establishes $f \in \Omega(g)$ as required.

An analogous argument establishes that the 2nd statement is also valid. The 3rd statement then follows from Propositions 3 and 4. Indeed, when $f \in \Theta(g) = \Omega(g) \cap \mathrm{O}(g)$ the first two statements apply and thus $\Theta(f) = \Omega(f) \cap \mathrm{O}(f) \subseteq \Omega(g) \cap \mathrm{O}(g) = \Theta(g)$, but we also have $g \in \Theta(f)$ hence $\Theta(g) \subseteq \Theta(f)$ for the same reason. □

Corollary 7. *For all complexity estimates f and g the following hold.*

(i) $\Omega(f) \subseteq \Omega(g)$ *if and only if* $\mathrm{O}(g) \subseteq \mathrm{O}(f)$

(ii) $\mathrm{O}(f) \subset \mathrm{O}(g)$ *if and only if* $\Omega(g) \subset \Omega(f)$

Proof. Consider the 1st statement. Proposition 6 gives $\Omega(f) \subseteq \Omega(g)$ if and only if $f \in \Omega(g)$ which is equivalent to $g \in \mathrm{O}(f)$ by Proposition 4 and hence also to $\mathrm{O}(g) \subseteq \mathrm{O}(f)$ using 6 one more time.

Consider the 2nd statement. Assuming that $\mathrm{O}(f) \subset \mathrm{O}(g)$ the 1st statement gives $\Omega(g) \subseteq \Omega(f)$ so we need only show that this inclusion is strict. By hypothesis there is $h \in \mathrm{O}(g)$ such that $h \notin \mathrm{O}(f)$, which by Proposition 4 are equivalent to $g \in \Omega(h)$ and $f \notin \Omega(h)$ respectively. Proposition 6 now gives $\Omega(g) \subseteq \Omega(h)$ and it follows that $f \notin \Omega(g)$. Since f is $\Omega(f)$ by Proposition 5 it witness that the we have a strict inclusion $\Omega(g) \subset \Omega(f)$. A similar argument justifies the opposite implication. □

Algebraic Compatibility

It is well known that we can add and multiply any two real valued functions f and g pointwise. More specifically, we define $f+g$ as the function $n \mapsto f(n)+g(n)$ and define $f \cdot g$ as the function $n \mapsto f(n) \cdot g(n)$. The complexity classes are well-behaved with respect to these operations.

Proposition 8 (Addition)**.** *For all complexity estimates e, f, g, and h the following hold.*

(i) If e is $\Omega(f)$ and h is $\Omega(g)$ then $e+h$ is $\Omega(f+g)$

(ii) If e is $\mathrm{O}(f)$ and h is $\mathrm{O}(g)$ then $e+h$ is $\mathrm{O}(f+g)$

(iii) If e is $\Theta(f)$ and h is $\Theta(g)$ then $e+h$ is $\Theta(f+g)$

Proof. Consider the 2nd statement. By hypothesis $e \in \mathrm{O}(f)$ hence there is $c' > 0$ and $n_0' > 0$ such that $0 \leq e(n) \leq c' \cdot f(n)$ for all $n \geq n_0'$. In addition $g \in \mathrm{O}(g)$ hence there is $c'' > 0$ and $n_0'' > 0$ such that $0 \leq h(n) \leq c'' \cdot g(n)$ for all $n \geq n_0'$. Fixing $c = \max\{c', c''\} > 0$ it follows that

$$0 \leq e(n) + h(n) \leq c' \cdot f(n) + c'' \cdot g(n) \leq c \cdot f(n) + c \cdot g(n) = c \cdot (f(n) + g(n))$$

for all $n \geq n_0 = \max\{n_0', n_0''\}$, since addition is monotonic and both $f(n)$ and $g(n)$ are non-negative. Thus we have witnesses n_0 and c showing that $(e+h) \in \mathrm{O}(f+g)$.

An analogous argument validates the 1st statement. The 3rd statement then follows from Proposition 3, because when both $e \in \Theta(f) = \Omega(f) \cap \mathrm{O}(f)$ and $h \in \Theta(g) = \Omega(g) \cap \mathrm{O}(g)$ hold the first two statements apply and thus $(e+h) \in \Omega(f+g) \cap \mathrm{O}(f+g) = \Theta(f+g)$. □

Proposition 9 (Multiplication)**.** *For all complexity estimates e, f, g, and h the following hold.*

(i) If e is $\Omega(f)$ and h is $\Omega(g)$ then $e \cdot h$ is $\Omega(f \cdot g)$

(ii) If e is $\mathrm{O}(f)$ and h is $\mathrm{O}(g)$ then $e \cdot h$ is $\mathrm{O}(f \cdot g)$

(iii) If e is $\Theta(f)$ and h is $\Theta(g)$ then $e \cdot h$ is $\Theta(f \cdot g)$

Proof. Consider the 1st statement. By hypothesis $e \in \Omega(f)$ hence there is $c' > 0$ and $n_0' > 0$ such that $0 \leq c' \cdot f(n) \leq e(n)$ for all $n \geq n_0'$. In addition $g \in \Omega(g)$ hence there is $c'' > 0$ and $n_0'' > 0$ such that $0 \leq c'' \cdot g(n) \leq h(n)$ for all $n \geq n_0'$. Fixing $c = \min\{c', c''\} > 0$ it follows that

$$0 \leq c^2 \cdot (f(n) \cdot g(n)) = c \cdot f(n) \cdot c \cdot g(n) \leq c' \cdot f(n) \cdot c'' \cdot g(n) \leq e(n) \cdot h(n)$$

for all $n \geq n_0 = \max\{n_0', n_0''\}$, since multiplication is monotonic and both $f(n)$ and $g(n)$ are non-negative. Thus we have witnesses n_0 and c^2 showing that $(e \cdot h) \in \Omega(f \cdot g)$.

An analogous argument validates the 2nd statement. The 3rd statement then follows from Proposition 3, because when both $e \in \Theta(f) = \Omega(f) \cap \mathrm{O}(f)$ and $h \in \Theta(g) = \Omega(g) \cap \mathrm{O}(g)$ hold the first two statements apply and thus $(e \cdot h) \in \Omega(f \cdot g) \cap \mathrm{O}(f \cdot g) = \Theta(f \cdot g)$. □

The behaviour of the complexity classes with respect to addition and multiplication of estimates is supplemented by the following closure properties. Their proofs of which are left as an exercise.

Proposition 10 (Scaling Closure). *For all real values $a > 0$ and complexity estimates f and g the following hold.*

(i) If f is $\Omega(g)$ then $a \cdot f$ is $\Omega(g)$

(ii) If f is $\mathrm{O}(g)$ then $a \cdot f$ is $\mathrm{O}(g)$

(iii) If f is $\Theta(g)$ then $a \cdot f$ is $\Theta(g)$

Proposition 11 (Additive Closure). *For all complexity estimates e, f, and g the following hold.*

(i) If both e and f are $\Omega(g)$ then $e + f$ is $\Omega(g)$

(ii) If both e and f are $\mathrm{O}(g)$ then $e + f$ is $\mathrm{O}(g)$

(iii) If both e and f is $\Theta(g)$ then $e + f$ is $\Theta(g)$

Exercise Activities

The following exercises will demonstrate your understanding of the theoretical aspects of algorithmic complexity classes.

Exercise 8. Prove that each Ω, O, and Θ complexity class is closed under multiplication by a positive scalar as stated in Proposition 10.

Exercise 9. Prove that each Ω, O, and Θ complexity class is closed under addition as stated in Proposition 11.

Exercise 10. Suppose e, f, and $f - e$ are all complexity estimates. Consider each the following statements about the complexity classes of an estimate g.

(i) If both e and f are $\Omega(g)$ then $f - e$ is $\Omega(g)$

(ii) If both e and f are $\mathrm{O}(g)$ then $f - e$ is $\mathrm{O}(g)$

If the statement is true, then prove it holds. If the statement is false, then show this is the case with a counter example.

Exercise 11. Consider each the following statements about complexity estimates e, f, g, and h.

(i) If e is $\Omega(f)$ and h is $\Omega(g)$ then h^e is $\Omega(g^f)$

(ii) If e is $\mathrm{O}(f)$ and h is $\mathrm{O}(g)$ then h^e is $\mathrm{O}(g^f)$

If the statement is true, then prove it holds. If the statement is false, then show this is the case with a counter example.

We now consider two procedures which underpin recursive sorting algorithms with good performance.

Example 5 (Quick Partition)**.** The following C language code implements an algorithm that rearranges the subarray between positions `lhs` to `rhs` of an array of integers `A`, so that those values smaller than or equal to the pivot `A[piv]` appear before it, and those greater than the pivot are placed after it. It returns the new position of the pivot value in the array.

```
int quick_partition(int *A, int lhs, int piv, int rhs) {
  if (piv != rhs) {
    int tmp = A[rhs];
    A[rhs] = A[piv];
    A[piv] = tmp;
  }
  int i = lhs-1;
  int j = rhs;
  while (i < j) {
    do {
      i++;
    } while (i < j && A[i] <= A[rhs]);
    do {
      j--;
    } while (j > i && A[j] >= A[rhs]);
    if (i < j) {
      int tmp = A[j];
      A[j] = A[i];
      A[j] = tmp;
    }
  }
  if (i != rhs) {
    int tmp = A[rhs];
    A[rhs] = A[i];
    A[i] = tmp;
  }
  return i;
}
```

The implementation temporarily places the pivot value at the end of the subarray. It proceeds by contracting the subarray window, scanning left-to-right for values greater than the pivot, scanning right-to-left for values smaller than the pivot, and swapping any such pair. When scanning ends, if the window is not at the end of the subarray then it starts with a value strictly greater than the pivot, and these two get swapped.

Exercise 12. Using best and worst case analyses develop lower and upper bound estimates for the running time of the quick partition algorithm in Example 5 in terms of the size n of the subarray; without ignoring any constant number of steps. Determine the appropriate tight asymptotic bound and justify this by applying the definition of the complexity classes to your estimates.

Example 6 (Sorted Merge)**.** The following C language code implements an algorithm that merges two contiguous subarrays of an array of integers `A` in ascending order; the left subarray occupies positions `lhs` to `cut`, and the right subarray positions `cut+1` to `rhs`.

```
void sorted_merge(int *A, int lhs, int cut, int rhs) {
  int len = rhs - lhs + 1;
  int B[len];
  int i = lhs;
  int j = cut+1;
  for (int k = 0; k < len; k++) {
    if (j > rhs) {
      B[k] = A[i]
      i++;
    } else if (i > cut) {
      B[k] = A[j]
      j++;
    } else if (A[j] > A[i]) {
      B[k] = A[i]
      i++;
    } else {
      B[k] = A[j]
      j++;
    }
  }
  for (int k = lhs; k < rhs+1; k++)
    A[k] = B[k];
}
```

The implementation creates a temporary array `B` to hold the values of `A` in positions `lhs` to `rhs` in ascending order, before updating `A` with the sorted subsequence.

You can watch sample runs of the merge algorithm in the Comp26120 videos for the complexity topic.

Exercise 13. Using best and worst case analyses develop lower and upper bound estimates for the running time of the sorted merge algorithm from Example 6 in terms of the total size n of the two subarrays; without ignoring any constant number of steps. Apply the definition of the complexity classes to your estimates in order to show that sorted merge is $\Theta(n)$.

Complexity Hierarchy

Once we have an upper or lower estimate $T(n)$ for the running time of an algorithm in terms of its input size $n \in \mathbb{N}$, we can use it to calculate the largest input that the algorithm can process in a given time interval.

Suppose our random-access machine model has a word size of 64 bits and its primitive instructions take 1 microsecond to execute. This means that a running time estimate $T(n)$ gives the number of microseconds that a set algorithm takes to process $64\,n$ bits of input data. Thus for a given time interval of Δt the equation

$$T(16384\,n) = T\left(\frac{1024^2}{64}\,n\right) = 10^6 \cdot \Delta t$$

can be solved for n to obtain the maximum number of (binary) megabits the algorithm can process in Δt seconds. For example, when $T(n) = n \log n$ and $\Delta t = 1$ second we find

$$\begin{aligned}
16384\,n\,\log(16384\,n) = 10^6 \cdot 1 \quad &\Longleftrightarrow \quad 16384\,n\,\ln(16384\,n) = 10^6 \cdot \ln 2 \\
&\Longleftrightarrow \quad \exp(\ln(16384\,n))\ln(16384\,n) = 10^6 \cdot \ln 2 \\
&\Longleftrightarrow \quad \mathrm{W}_0(\exp(\ln(16384\,n))\ln(16384\,n)) = \mathrm{W}_0\left(10^6 \ln 2\right) \\
&\Longleftrightarrow \quad \ln(16384\,n) = \mathrm{W}_0\left(10^6 \ln 2\right) \\
&\Longleftrightarrow \quad 16384\,n = \exp\left(\mathrm{W}_0\left(10^6 \ln 2\right)\right) \\
&\Longrightarrow \quad n \approx 3.83
\end{aligned}$$

So the algorithm would be able to process close to 4 megabits of input per second in this example. Notice that this kind of calculation can result in hard equations which require the use of sophisticated complex analysis operators to solve. Indeed, the form of the above equation was $x \log x = c$ whose solution for $x \geq 0$ involves the principal branch W_0 of the Lambert W function[1].

The following table has values of such performance metrics (words per second, words per minute, words per hour) for common complexity estimates.

Δt	$\log n$	$\sqrt{n}$	n	$n \log n$	n^2	n^3	2^n	$n!$
1 second	$2^{1 \cdot 10^6}$	$1 \cdot 10^{12}$	$1 \cdot 10^6$	62746	1000	100	19	9
1 minute	$2^{6 \cdot 10^7}$	$3.6 \cdot 10^{15}$	$6 \cdot 10^7$	2801417	7745	391	25	11
1 hour								

(12)

The complexity classes offer an alternative to the calculation of performance values for algorithms that we'd like to compare. In particular, when we have a strict inclusion $\mathrm{O}(f) \subset \mathrm{O}(g)$ we can be certain that the performance of f is better than that of g for sufficiently large input sizes. To this end we now formally establish an important part of the hierarchy of complexity classes.

[1] You can use Python's SciPy library to calculate Lambert W_0 values. This is how the result for the above example can be obtained with Python on the console

```
>>> import numpy as np
>>> from scipy.special import lambertw
>>> lambertw(10**6 * np.log(2))
(11.04685212552196+0j)
>>> np.exp( lambertw(10**6 * np.log(2)) )/16384
(3.82971963315968+0j)
>>>
```

We start with some technical results about polynomial and exponential growth.

Proposition 13 (Polynomial). *For all natural numbers k and j the following hold.*

(i) If $j < k$ then $\mathrm{O}(n^j) \subset \mathrm{O}(n^k)$ and also $\Omega(n^k) \subset \Omega(n^j)$

(ii) Every polynomial complexity estimate f of degree k is $\mathrm{O}(n^k)$

Proof. Consider the 1st statement. Since $k \geq j+1$ and exponentiation of a positive base is monotonically increasing, it is clear that $0 \leq n^j \leq n^j \cdot n \leq n^k$ for positive values $n \geq 1$. Thus $c = 1$ and $n_0 = 1$ witness that n^j and n^{j+1} are both $\mathrm{O}(n^k)$. Proposition 6 now justifies that $\mathrm{O}(n^j) \subseteq \mathrm{O}(n^k)$. We now show that this inclusion is strict by establishing that $n^{j+1} \notin \mathrm{O}(n^j)$. Assume by contradiction that n^{j+1} is $\mathrm{O}(n^j)$, and hence there is $c' > 0$ and $n'_0 > 0$ such that for all $n \geq n'_0$ we have

$$0 \leq n^{j+1} \leq c' \cdot n^j \quad \Longrightarrow \quad n \leq c'$$

as we are allowed to divide the inequality by $n^j > 0$. It is obvious that $n \leq c'$ does not hold when $n > \max\{c', n'_0\}$ so the assumption is false and $n^{j+1} \notin \mathrm{O}(n^j)$. This establishes that $\mathrm{O}(n^j) \subset \mathrm{O}(n^k)$ which by Corollary 7 is equivalent to $\Omega(n^k) \subset \Omega(n^j)$.

Now consider the 2nd statement. Since polynomial coefficients may take any real value, it is convenient to consider the more general statement:

For each real polynomial function $f(n)$ of degree k there is a constant $c > 0$ such that $c \cdot n^k$ eventually dominates f; meaning that $f(n) \leq c \cdot n^k$ for some r and all $n \geq r$. (14)

Indeed, if all polynomials of degree k are eventually dominated by a positive scaling of n^k, then in particular every polynomial complexity estimate $f(n)$ of degree k is $\mathrm{O}(n^k)$ as the proposition states, since every estimate satisfies $0 \leq f(n)$ for all domain values n. Note that we are allowed to use any real value for the witness r of eventual domination domination, because we can always replace it by $n_0 = \max\{\lceil r \rceil, 1\}$ to satisfy the constraints of the big-O complexity class.

We proceed by induction on k. The base case is when $k = 0$ and hence $f(n) = a$ is constant. Setting $c = \max\{a, 1\} > 0$ we always have $a \leq c \cdot 1$, hence to confirm that $c \cdot n^0$ eventually dominates f we may fix $r = 1$. Assume statement (14) is true as our induction hypothesis. Now consider a polynomial function f' of degree $k+1$, thus $f'(n) = a_{k+1} n^{k+1} + f(n)$ for some polynomial f of degree k. By hypothesis $f(n) \leq c \cdot n^k$ for some $c > 0$ and all n greater or equal to some value r. Setting $c' = \max\{c, a_{k+1}\} > 0$ and $r' = \max\{r, 1\} > 0$ it follows that

$$f'(n) = a_{k+1} n^{k+1} + f(n) \leq c' \cdot n^{k+1} + c' \cdot n^k \leq c' \cdot n^{k+1} + c' \cdot n^{k+1} = 2\,c' \cdot n^{k+1}$$

for all $n \geq r'$, where the second inequality holds because c' and n are strictly positive. Note that we allow the coefficient a_{k+1} and the value of $f(n)$ to be negative, but $2\,c'$ and r' are strictly positive constants witnessing that $2\,c' \cdot n^{k+1}$ eventually dominates $f'(n)$. Moreover, by induction the statement (14) holds and it justifies the result. □

Proposition 15 (Exponential Growth). *For all natural numbers k and n the following hold.*

(i) $n < 2^n$ moreover $\log n < n$ if $n > 0$.

(ii) If $n > 2^2$ then $n^2 < 2^n$.

(iii) If $k > 1$ and $n > k^2$ then $n^k < 2^n$.

Proof. Consider the 1st statement. Note that if $n > 0$ then $n < 2^n$ entails $\log n < \log 2^n = n$ as log is strictly increasing; thus it suffices to prove the former. We proceed by induction on n. When $n = 0$ it is clear that $0 < 1 = 2^0$ and if $n = 1$ we also confirm $1 < 2 = 2^1$. Assuming $n < 2^n$ as our induction hypothesis, for $n \geq 1$ it follows that

$$n + 1 \leq n + n = n \cdot 2 < 2^n \cdot 2 = 2^{n+1}$$

Moreover, by induction the 1st statement holds for all $n \in \mathbb{N}$.

We justify the 2nd statement by induction on n. The base case is when $n = 5$ and it is clear that $5^2 = 25 < 32 = 2^5$. Assuming $n^2 < 2^n$ as our induction hypothesis, for $n \geq 5$ it follows that

$$(n+1)^2 = n^2\left(1+\frac{1}{n}\right)^2 \leq n^2\left(1+\frac{1}{5}\right)^2 < 2^n \cdot 2 = 2^{n+1}$$

Moreover, by induction the 2nd statement holds for $n \geq 5$.

In order to justify the last statement we need the following auxiliary result.

$$k > 0, n > 1 \quad \Longrightarrow \quad \left(1-\frac{k}{n}\right)\left(1+\frac{1}{n}\right)^k < 1 \tag{16}$$

We proceed by induction on k. The base case is when $k = 1$ and we confirm that

$$\left(1-\frac{1}{n}\right)\left(1+\frac{1}{n}\right) = 1 - \frac{1}{n^2} < 1$$

where the inequality arises because $1/n^2 > 0$ if $n > 1$. Assuming (16) holds for some k and all $n > 1$ as our induction hypothesis, it follows that

$$\begin{aligned}\left(1-\frac{k+1}{n}\right)\left(1+\frac{1}{n}\right)^{k+1} &= \left(1-\frac{k+1}{n}\right)\left(1+\frac{1}{n}\right)\left(1+\frac{1}{n}\right)^k \\ &= \left(1-\frac{k}{n}-\frac{k+1}{n^2}\right)\left(1+\frac{1}{n}\right)^k \quad < \left(1-\frac{k}{n}\right)\left(1+\frac{1}{n}\right)^k < 1\end{aligned}$$

where the first inequality arises because $(k+1)/n^2 > 0$ when both $k > 0$ and $n > 1$. This justifies (16) by induction. As a consequence of this result, for $n > 2k$ we have

$$\frac{1}{2}\left(1+\frac{1}{n}\right)^k = \left(1-\frac{k}{2k}\right)\left(1+\frac{1}{n}\right)^k < \left(1-\frac{k}{n}\right)\left(1+\frac{1}{n}\right)^k < 1$$

Moreover this establishes

$$k > 0, n > 2k \quad \Longrightarrow \quad \left(1+\frac{1}{n}\right)^k < 2 \tag{17}$$

We can now consider the 3rd statement. Note that if $k = 2$ then this statement reduces to the 2nd. In addition, the conditions $k > 1$ and $n > k^2$ ensure that $n > k^2 \geq 2k$ and thus provide the requirements of (17). We proceed by induction on n. The base case is when $n = k^2 + 1$ and we confirm that

$$(k^2+1)^k = (k^2)^k\left(1+\frac{1}{k^2}\right)^k < (2^k)^k\left(1+\frac{1}{k^2}\right)^k < (2^k)^k \cdot 2 = 2^{(k^2+1)}$$

where the first inequality is justified by the 2nd statement, and the second by (17). Assuming the 3rd statement holds for some n and all $k > 1$ as our induction hypothesis, it follows that

$$(n+1)^k = n^k\left(1+\frac{1}{n}\right)^k < 2^n\left(1+\frac{1}{n}\right)^k < 2^n \cdot 2 = 2^{(n+1)}$$

where (17) validates the second inequality. Moreover, by induction the 3rd statement holds for $k > 1$ and $n \geq k^2$. This completes the proof of the proposition. $\square$

The following result allows us to compare logarithmic, polynomial, and exponential asymptotic behaviour.

Proposition 18 (Hierarchy). *For all natural numbers $k \geq 1$ the big-Ω and big-O classes make the following strict complexity hierarchy.*

$$\Omega(2^n) \subset \Omega(n^k) \subset \Omega(\log n) \subset \Omega(1)$$

$$\mathrm{O}(1) \subset \mathrm{O}(\log n) \subset \mathrm{O}(n^k) \subset \mathrm{O}(2^n)$$

Proof. The statement about the big-Ω hierarchy is equivalent to the one for the big-O hierarchy as a consequence of Corollary 7. Indeed, this result establishes that

$$\begin{aligned} \Omega(2^n) \subset \Omega(n^k) \quad &\iff \quad \mathrm{O}(n^k) \subset \mathrm{O}(2^n) \\ \Omega(n^k) \subset \Omega(\log n) \quad &\iff \quad \mathrm{O}(\log n) \subset \mathrm{O}(n^k) \\ \Omega(\log n) \subset \Omega(1) \quad &\iff \quad \mathrm{O}(1) \subset \mathrm{O}(\log n) \end{aligned}$$

thus we need only justify three of these six class inclusions. We treat the required cases separately.

Case $\Omega(\log n) \subset \Omega(1)$**:**
Since log is increasing, if $2 \leq n$ then $1 = \log 2 \leq \log n$. Thus setting $c = 1$ and $n_0 = 2$ we have $0 \leq c \cdot 1 \leq \log n$ for all $n \geq n_0$. This means $\log n \in \Omega(1)$ and using Proposition 6 also $\Omega(\log n) \subseteq \Omega(1)$. We now show that $1 \notin \Omega(\log n)$ which establishes that the inclusion is strict, as 1 is $\Omega(1)$ by Proposition 5.

Assume by contradiction that 1 is $\Omega(\log n)$, and hence there is $c' > 0$ and $n_0' > 0$ such that $0 \leq c' \cdot \log n \leq 1$ for all $n \geq n_0'$. It is necessary that $\log n \leq 1/c'$ but this does not hold when $n > 2^{n'}$ where $n' = \max\{n_0', \lceil 1/c' \rceil\}$. We must conclude that the assumption is false and $1 \notin \Omega(\log n)$ as required.

Case $\Omega(n^k) \subset \Omega(\log n)$**:**
To show that this inclusion holds for all $k \geq 1$ we need only establish $\Omega(n) \subset \Omega(\log n)$ as Proposition 13 gives $\Omega(n^k) \subset \Omega(n)$ for all $k > 1$. Setting $c = 1$ and $n_0 = 2$, the 1st part of Proposition 15 validates $0 \leq c \cdot \log n \leq n$ for all $n \geq n_0$. This means $n \in \Omega(\log n)$ and using Proposition 6 also $\Omega(n) \subseteq \Omega(\log n)$. We now show that $\log n \notin \Omega(n)$ which establishes that the inclusion is strict, as $\log n$ is $\Omega(\log n)$ by Proposition 5.

Assume by contradiction that $\log n$ is $\Omega(n)$, and hence there is $c' > 0$ and $n_0' > 0$ such that $0 \leq c' \cdot n \leq \log n$ for all $n \geq n_0'$. It is necessary that $c' < 1$ since $c' \cdot n_0' \leq \log n_0' < n_0'$ using the 1st part of Proposition 15 and the fact that $n_0' \geq 1$. Now consider $n' = \max\{n_0', 5, \lceil 1/c' \rceil\}$. Since $n' \geq n_0'$ by hypothesis we must have

$$2^{n'} \leq (1/c') \cdot \log 2^{n'} = (1/c') \cdot n' \leq n' \cdot n' = (n')^2$$

where the second inequality holds because $n' \geq \lceil 1/c' \rceil$. However, as $n' \geq 5$ this now contradicts the 2nd part of Proposition 15. We must conclude that the assumption is false and $\log n \notin \Omega(n)$ as required.

Case $\Omega(2^n) \subset \Omega(n^k)$**:**
Note that we need to show this inclusion holds for each $k \geq 1$, but we are allowed to vary the witnesses of the argument with the value of k. Setting $c = 1$ and $n_0 = k^2 + 1$, we find that $0 \leq c \cdot n^k \leq 2^n$ for all $n \geq n_0$ using the 3rd part of Proposition 15 if $k > 1$ or it's 1st part when $k = 1$. This means $2^n \in \Omega(n^k)$ and using Proposition 6 also $\Omega(2^n) \subseteq \Omega(n^k)$. We now show that $n^k \notin \Omega(2^n)$ which establishes that the inclusion is strict, as 2^n is $\Omega(2^n)$ by Proposition 5.

Assume by contradiction that n^k is $\Omega(2^n)$, and hence there is $c' > 0$ and $n_0' > 0$ such that $0 \leq c' \cdot 2^n \leq n^k$ for all $n \geq n_0'$. It is necessary that $c' < 1$ since $c' \cdot 2^{n_0'} \leq (n_0')^k < 2^{n_0'}$

using the 1st or 3rd part of Proposition 15 depending on the value of $k > 1$. Now consider $n' = \max\{n'_0, (k+1)^2 + 1, \lceil 1/c' \rceil\}$. Since $n' \geq n'_0$ by hypothesis we must have

$$2^{n'} \leq (1/c') \cdot (n')^k \leq n' \, (n')^k = (n')^{k+1}$$

where the second inequality holds because $n' \geq \lceil 1/c' \rceil$. However, as $n' \geq (k+1)^2 + 1$ this now contradicts the 3rd part of Proposition 15. We must conclude that the assumption is false and $n^k \notin \Omega(2^n)$ as required.

Moreover, all the class inclusions in the stated hierarchy are justified. □

Further Exercises

Exercise 14. Confirm the performance values of the complexity estimates given in table (12), and calculate the missing words per hour performance values. (Note that you can also use trial and error to solve hard equations.)

Exercise 15. John is working as a software developer at BoostCode UK Ltd. He needs to compare two implementations of sorting algorithms on the same machine, which will be later integrated into a software product. In particular, for inputs defined by size n, the first algorithm denoted by A runs in $4n^2$, while the second algorithm denoted by B runs in $128n \log n$. John needs to compute values of n where algorithm A beats algorithm B so that he can make the best choice of which one will be integrated into the product. For which values of n will algorithm A run faster than algorithm B?

Exercise 16. Suppose that the function `jumble`, which manipulates three chosen positions of a given array, always runs in constant time s. Consider the following C language code that manipulates an array `A` of size `n`.

```
void wheel(int *A, int n) {
  for (int i = n; i > 0; i /= 2)
    for (int j = 1; j < n; j *= 2)
      for (int k = 0; k < n; k += 2)
        jumble(A, i, j, k);
}
```

What is the big-Θ time complexity of the `wheel` function?

Exercise 17. Suppose that the function `randomInt` takes a constant number s of computational steps to produce a random integer number, and that the function `newSort` implements a sorting algorithm for an array `A` with `n` integer values with complexity $\mathrm{O}(n \log n)$. Consider the following C language code that tests the `newSort` implementation.

```
void test(int n) {
  int A[n];
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++)
      A[j] = randomInt(i);
    newSort(A, n);
  }
}
```

What is the big-O time complexity of the `test` function?

Exercise 18. Consider each the following statements.

(i) The estimate 2^{n+1} is $\mathrm{O}(2^n)$.

(ii) The estimate 2^{2n} is $\mathrm{O}(2^n)$.

If the statement is true, then prove it holds. If the statement is false, then show this is the case.

Exercise 19. Prove the following statements for all natural numbers $k \geq 1$.

(i) The estimate $\log(n^k)$ is $\mathrm{O}(\log n)$.

(ii) The estimate n^k is $\mathrm{O}(a^n)$ for any fixed real value $a > 1$.

Exercise 20. Prove the following strict inclusions of the complexity hierarchy.

(i) $\Omega(n!) \subset \Omega(2^n)$

(ii) $\mathrm{O}(2^n) \subset \mathrm{O}(n!)$

A. Some Mathematical Background

This section highlights some relevant mathematical materials for the algorithmic complexity topic on Comp26120. You can find more mathematical background in the appendices of the recommended textbooks. Note that you are expected to have already studied these topics in the 1st year of the degree and in secondary education (although the following presentation may be a bit more abstract).

Definition 19. Let $f\colon X \to A$ be a function between two subsets of $\mathbb{R}$. We say that f is *monotonically increasing* just in case

$$x \leq y \quad \Longrightarrow \quad f(x) \leq f(y)$$

for all x and y in X. We say that f is *monotonically decreasing* precisely when

$$x \leq y \quad \Longrightarrow \quad f(y) \leq f(x)$$

for all x and y in X. The notions of *strictly increasing* and *strictly decreasing* are similarly defined but using a strict inequality; i.e. replacing $\leq$ with $<$.

The following derivative tests are a consequence of the *Mean Value Theorem* and often useful to study the monotonicity of real valued functions.

Proposition 20 (Derivative Test)**.** *Suppose the function $f\colon [a,b] \to \mathbb{R}$ is differentiable on the open interval (a,b) and let f' denote its derivative and $c \in [a,b]$.*

(i) If $f'(x) \geq 0$ for all $x \in (a,b)$ then f is monotonically increasing on $[a,b]$

(ii) If $f'(x) \leq 0$ for all $x \in (a,b)$ then f is monotonically decreasing on $[a,b]$

(iii) If $f'(x) > 0$ for all $x \in (a,b)$ then f is strictly increasing on $[a,b]$

(iv) If $f'(x) < 0$ for all $x \in (a,b)$ then f is strictly decreasing on $[a,b]$

(v) If $f'(c) = 0$ with f strictly increasing on (a,c) and strictly decreasing on (c,b), then f has a local maximum at c

(vi) If $f'(c) = 0$ with f strictly decreasing on (a,c) and strictly increasing on (c,b), then f has a local minimum at c

Proposition 21. *For all real values a, b, and c exponentiation satisfies the following.*

(i) $b^0 = 1$ *but is undefined when* $b = 0$

(ii) $b^1 = b$

(iii) $b^{-a} = 1/b^a$ *but is undefined when* $b = 0$

(iv) $b^{1/a} = \sqrt[a]{b}$ *but is undefined when* $b < 0$ *or* $a = 0$

(v) $b^a \cdot b^c = b^{a+c}$ *if both sides are defined*

(vi) $(b^a)^c = b^{a \cdot c}$ *if both sides are defined*

(vii) $b^a \cdot c^a = (b \cdot c)^a$ *if both sides are defined*

Definition 22. The logarithmic function is defined by

$$\log_b c = a \quad \Longleftrightarrow \quad b^a = c$$

for all real values a, $b > 0$ provided $b \neq 1$, and $c > 0$.

Proposition 23. *For all positive real values a, $b \neq 1$, and c logarithms satisfy the following.*

(i) $\log_b 1 = 0$

(ii) $\log_b b = 1$

(iii) $\log_b (1/c) = -\log_b c$

(iv) $\log_b \sqrt[a]{c} = \log_b c/a$

(v) $\log_b (a \cdot c) = \log_b a + \log_b c$

(vi) $\log_b c^a = a \cdot \log_b c$

Proposition 24 (Changing Base)**.** *For all positive real values a, $b \neq 1$, and c*

(i) $b^{\log_b c} = c$

(ii) $a^{\log_b c} = c^{\log_b a}$

(iii) $\log_b c = \log_a c / \log_a b$ *provided* $a \neq 1$

In Computer Science we assume the logarithmic base $b = 2$ when not specified. This means that $\log c = \log_2 c$ which is known as the *binary logarithm.* Note that in other physical sciences and *almost all calculators* the assumption is the base $b = 10$ if unspecified, which is known as the *common logarithm.* Also note that Euler's number e is the base for the *natural logarithm*, usually written $\ln c = \log_{\mathrm{e}} c$. The changing base result allows us to easily obtain binary logarithms on a calculator using one of the following translations

$$\log c = \frac{\log_{10} c}{\log_{10} 2} = \frac{\ln c}{\ln 2}$$

Proposition 25. *Fix a positive real value b. The function $a \mapsto b^a$ defined for $a \in \mathbb{R}$*

(i) has derivative $b^a \ln b$, and

(ii) is strictly increasing when $b > 1$.

Proposition 26. *Fix a positive real value b. The function $c \mapsto \log_b c$ defined for $c > 0$*

(i) has derivative $1/(c \ln b)$, and

(ii) is strictly increasing.

Given a finite sequence of n values $(a_1, a_2, \ldots, a_n)$ the sum of its terms $a_1 + a_2 + \cdots + a_n$ is written $\sum_{i=1}^{n} a_i$. The sum of an infinite sequence $(a_1, a_2, \ldots, a_n, \ldots)$ is defined as the limit $\lim_{n\to\infty} \sum_{i=1}^{n} a_i$.

Definition 27. An *arithmetic progression* (a_i) is a sequence of values $(a_1, a_2, \ldots)$ with a constant difference between any two consecutive terms. The constant $d = a_{i+1} - a_i$ is known as the *common difference* of the progression.

Proposition 28. *If (a_i) is an arithmetic progression with common difference d, then it can be written in closed form as $a_i = a_1 + d(i-1)$. Moreover, the sum of its first n terms is given by*

$$\sum_{i=1}^{n} a_i = \frac{n}{2} \cdot (a_1 + a_n)$$

Definition 29. A *geometric progression* (a_i) is a sequence of values $(a_1, a_2, \ldots)$ with a constant ratio between any two consecutive terms. The constant $r = a_{i+1}/a_i$ is known as the *common ratio* of the progression.

Proposition 30. *If (a_i) is a geometric progression with common ratio r, then it can be written in closed form as $a_i = a_1 \cdot r^{(i-1)}$. Moreover, the sum of its first n terms is given by*

$$\sum_{i=1}^{n} a_i = \begin{cases} a\,(n+1) & \text{if } r = 1 \\ a\left(\dfrac{1 - r^{(n+1)}}{1 - r}\right) & \text{otherwise} \end{cases}$$